CSI101 COMPUTER SYSTEMS

Lab 3 — ECX as an Automatic Loop Counter

Assembly Programming · Practical application 11.3 — ECX as an automatic loop counter

2-hour lab session · Visual Studio 2022 · Win32 Console Application · x86 platform

The `LOOP` instruction decrements `ECX` and jumps back in a single instruction. It is convenient, it has two traps that catch everyone once, and it is the natural way to walk along an array. This lab covers all three.

What you will be able to do

- Use `LOOP` to build a counted loop with no explicit comparison and no explicit decrement.
- Walk along a C++ array using a pointer register, stepping four bytes at a time.
- Use `TEST` to ask about individual bits rather than to order two values.
- Maintain two running results inside one pass over the data.
- Recognise the two situations in which `LOOP` misbehaves.

How the session is organised

0:00 – 0:25 Part A. Type the lecture program in, build it, run it, and read the notes.

0:25 – 1:10 Task 1. Extend the program.

1:10 – 1:50 Task 2. Extend it again.

1:50 – 2:00 Work through the checklist at the end and ask about anything that did not build.

> **TYPE IT, DO NOT PASTE IT**
>
> Typing catches the syntax details that reading skips over — the missing comma, the wrong register width, the label with no instruction after it. The solutions at the end of this handout are for checking your work, not for copying.

Before you start

1. Open Visual Studio 2022 and choose **File > New > Project > Console App (C++)**. Give it the name shown at the top of this handout.
2. Set the **Solution Platforms** dropdown on the toolbar to **x86**. It defaults to x64, where the `_asm` keyword is rejected outright with error C4235.
3. Replace the generated `main` with the Part A listing, and make sure `#include <cstdio>` is at the top — `printf` and `scanf_s` come from there.
4. Press **Ctrl + F5** to build and run without the debugger, so the console window stays open when the program finishes.

> **IF THE PLATFORM IS WRONG**
>
> With the platform left at x64 the compiler reports that the `_asm` keyword is not supported on this architecture (C4235). Inline assembly exists on 32-bit x86 and nowhere else.

Part A — the lecture program

Two uses of LOOP. The first adds the integers from 1 to 10 by counting down in ECX and adding the counter itself. The second walks along a five-element array, adding each element and stepping the pointer on by four bytes.

Declarations

```
#include <cstdio>

// A preprocessor macro, not a const int.  #define SIZE 5 becomes the plain
// number 5 by the time the assembler sees it, which is what "mov ecx, SIZE"
// needs.  A const int is a variable, and would be read from memory instead.
#define SIZE 5

int data[SIZE] = { 4, 8, 15, 16, 23 };
```

The program

```
void partA()
{
    int total = 0;
    int arrayTotal = 0;

    printf("--- Part A: LOOP as an automatic counter ---\n\n");

    // ---- sum the integers from 1 to 10 --------------------------------
    _asm {
        xor     eax, eax            // total = 0 (faster than mov eax, 0)
        mov     ecx, 10             // LOOP always counts down in ECX

    sumLoop:
        add     eax, ecx            // add the current counter value
        loop    sumLoop             // dec ecx, then jump back while ecx != 0

        mov     total, eax          // 10+9+8+...+1 = 55
    }

    // ---- sum the five elements of the array ---------------------------
    _asm {
        push    esi                 // ESI must be preserved across the block

        lea     esi, data           // esi = address of data[0]
        xor     eax, eax            // running total
        mov     ecx, SIZE           // five elements, so five iterations

    nextItem:
        add     eax, [esi]          // add the int that esi points at
        add     esi, 4              // step on by one int (4 bytes)
        loop    nextItem

        mov     arrayTotal, eax     // 4+8+15+16+23 = 66

        pop     esi                 // restore it before leaving
    }

    printf("  Sum of 1..10           = %d\n", total);
    printf("  Sum of the array       = %d\n", arrayTotal);
    printf("\nLOOP does two jobs at once: it decrements ECX and it jumps.\n");
}
```

Expected output

```
  Sum of 1..10           = 55
  Sum of the array       = 66
```

What to notice

- `LOOP` does two things at once: it decrements `ECX`, then jumps to the label if `ECX` is not zero. `ECX` is not optional — no other register can be used.
- `LOOP` sets **no flags**, so it is safe to place after a comparison whose flags you still care about.
- **Trap one:** if `ECX` is already 0 when the loop begins, `LOOP` decrements first, `ECX` wraps round to 4,294,967,295, and the loop runs about four billion times. Test for zero before you start.
- **Trap two:** `LOOP` can only make a short jump, so the target must be within 128 bytes. A long loop body produces a jump-out-of-range error at build time.
- `#define SIZE 5` is used rather than `const int`. A preprocessor macro becomes a plain number the assembler can put straight into `mov ecx, SIZE`; a `const int` is a variable, and would have to be read from memory.
- `ESI` is one of the registers you must preserve, so the array version pushes it on entry and pops it on exit.

Task 1 — add up only the even values

What to write

- Walk the same array with `LOOP`, but add a value to the running total only when it is even.
- Count how many even values there were.
- Print both the count and the sum.

Expected output

```
Array           = { 4, 8, 15, 16, 23 }
Even values     = 3
Sum of evens    = 28
```

Hints

- A number is even when its lowest bit is 0. `test edx, 1` performs a bitwise AND and keeps only the flags, so `JNZ` jumps when that bit was set — that is, when the value is odd.
- `TEST` is the right instruction here, not `CMP`. `CMP` orders two values; `TEST` asks whether particular bits are set.
- Give each register one job and write it down before you start: `ESI` the pointer, `EAX` the sum, `EBX` the count, `EDX` the current element, `ECX` the loop counter.
- You are borrowing both `ESI` and `EBX`, so push both at the top and pop them in the **reverse** order at the bottom.

Task 2 — largest and smallest in a single pass

What to write

- Find both the largest and the smallest value in the array.
- Do it in one pass — a single `LOOP`, with two comparisons inside it.
- Print both results.

Expected output

```
Array           = { 4, 8, 15, 16, 23 }
Largest         = 23
Smallest        = 4
```

Hints

- Take element 0 as the starting guess for **both** answers, then compare the remaining four elements against them.
- Because element 0 is already accounted for, set `ECX` to `SIZE` and then `DEC ECX`, and step `ESI` on by 4 before the loop begins.
- That `DEC ECX` is exactly the situation trap one warns about. With a one-element array `ECX` would reach zero here, and `LOOP` would wrap. Think about where you would put the guard.
- Use signed jumps. If the array ever held negative values, `JBE` and `JAE` would give the wrong answer without any warning.

Before you leave

- The Solution Platform reads **x86**, not x64.
- Part A prints 55 and 66.
- `ESI` and `EBX` are pushed on entry and popped in reverse order on exit.
- The even-value task uses `TEST`, not `CMP`.
- You can explain what would happen if `ECX` were zero when a `LOOP` began.

If something will not build

- `_asm not supported on this architecture` — the platform is still x64. Change it to x86.
- `improper operand type` — the operand size is ambiguous. Add `DWORD PTR`, or check you have not mixed `eax` with `al`.
- `undeclared identifier` inside a block — a misspelt variable name, or one not in scope. Names are **case sensitive** inside `_asm`, even though instruction mnemonics are not.
- The program builds but crashes *after* the block — the stack is unbalanced, or `EBX`, `ESI`, `EDI` or `EBP` was not restored.
- Right answer, wrong sign — an unsigned jump was used on signed data, or `SHR` where `SAR` was needed.

Solutions

Complete, commented code for both tasks. The Part A listing earlier in this handout is already the full solution for that section. Everything here matches the `CSI101_Lab03_Solution` project in the accompanying Visual Studio package line for line.

Task 1 — add up only the even values

```
void task1()
{
    int evenSum   = 0;
    int evenCount = 0;

    printf("\n--- Task 1: sum only the even values ---\n\n");

    _asm {
        push    esi                 // preserve the registers we borrow
        push    ebx

        lea     esi, data           // esi = address of data[0]
        xor     eax, eax            // eax = running sum of even values
        xor     ebx, ebx            // ebx = how many even values we found
        mov     ecx, SIZE           // loop counter

    nextItem:
        mov     edx, [esi]          // edx = the current element

        test    edx, 1              // AND with 1, keep only the flags
        jnz     skipOdd             // ZF = 0 means the low bit was 1 -> odd

        add     eax, edx            // even, so add it in
        inc     ebx                 // and count it

    skipOdd:
        add     esi, 4              // move to the next int
        loop    nextItem            // LOOP changes no flags, so it is safe here

        mov     evenSum, eax
        mov     evenCount, ebx

        pop     ebx                 // restore in the REVERSE order
        pop     esi
    }

    printf("  Array           = { 4, 8, 15, 16, 23 }\n");
    printf("  Even values     = %d\n", evenCount);      // 3
    printf("  Sum of evens    = %d\n", evenSum);        // 28
}
```

Notes on this solution

- The `INC EBX` sits inside the even branch, so it counts only what was added. Moving it outside the branch would count every element — a mistake worth making once in the debugger to see how it looks.
- `ADD ESI, 4` appears after the label, so it runs on both paths. If it were inside the even branch the pointer would stop advancing at the first odd number and the loop would read the same element five times.

Task 2 — largest and smallest in a single pass

```
void task2()
{
    int largest  = 0;
    int smallest = 0;

    printf("\n--- Task 2: largest and smallest in one pass ---\n\n");

    _asm {
        push    esi
        push    ebx
```

```
        lea     esi, data
        mov     eax, [esi]          // assume element 0 is the largest ...
        mov     ebx, eax            // ... and also the smallest

        mov     ecx, SIZE
        dec     ecx                 // element 0 is already accounted for,
                                    //   so only SIZE-1 comparisons remain
        add     esi, 4              // start the scan at element 1

    scanLoop:
        mov     edx, [esi]          // edx = the candidate

        cmp     edx, eax
        jle     checkSmallest       // SIGNED jump - the data may be negative
        mov     eax, edx            // new largest

    checkSmallest:
        cmp     edx, ebx
        jge     nextOne
        mov     ebx, edx            // new smallest

    nextOne:
        add     esi, 4
        loop    scanLoop

        mov     largest, eax
        mov     smallest, ebx

        pop     ebx
        pop     esi
    }

    printf("  Array           = { 4, 8, 15, 16, 23 }\n");
    printf("  Largest         = %d\n", largest);        // 23
    printf("  Smallest        = %d\n", smallest);       // 4
    printf("\nNote the DEC ECX before the loop: if SIZE were 1, ECX would reach\n");
    printf("zero here and LOOP would wrap round to 4,294,967,295 iterations.\n");
}
```

Notes on this solution

- Both comparisons use the same candidate in `EDX`, which is why the value is loaded into a register first rather than being compared from memory twice.
- The `JLE` and `JGE` skips mean that ties leave the existing answer alone. That costs nothing here, but with an array of records it would decide which of two equal records you kept.